

Course	CSE 445 — Computer Vision
Instructor	Dr. Mohammad Rifat Ahmmad Rashid, Associate Professor, Dept. of CSE, EWU
Assignment Type	Group assignment (datasets pre-assigned per group)
Submission Deadline	July 12, 2026 (12.07.2026)
Assessment Mode	Viva voce examination — focused on code, parameter choices, and results analysis
Submission	Five public Kaggle notebook links posted on Google Classroom

1. Overview

This assignment is Part 1 of a two-part project on applied computer vision. Each group builds a complete object detection pipeline: dataset analysis, leakage-safe splitting, augmentation, and independent training, evaluation, and error analysis for four detector variants (YOLOv10, YOLOv12, YOLOv26, RF-DETR). Part 2 will be announced separately.

Each group submits **five Kaggle notebooks**: one for dataset exploration and preprocessing, and one dedicated notebook per model that covers training, evaluation, and error analysis end-to-end. All model-size choices (n/s/m/l/x for YOLO; nano/base for RF-DETR) are made by the group based on dataset characteristics, following the guidance in Section 4.

2. Learning Objectives

By the end of this assignment students should be able to:

- Perform structured EDA on an object detection dataset and extract actionable insights.
- Design train/validation/test splits that are free of data leakage.
- Select and justify augmentation strategies grounded in EDA findings.
- Choose an appropriate model variant size for a given dataset and justify the choice.
- Configure, train, and monitor YOLOv10, YOLOv12, YOLOv26, and RF-DETR within Kaggle's compute constraints.
- Evaluate models with standard detection metrics and conduct thorough error analysis.
- Articulate every code and hyperparameter decision in a live viva examination.

3. Dataset and Group Assignment

Each group has been assigned a distinct dataset. Use only that dataset across all five notebooks. Before starting, confirm:

- Dataset source/link, annotation format (COCO JSON, PASCAL VOC XML, or YOLO TXT), class list, and number of images.
- Whether official train/valid/test splits are provided (groups must still verify them for leakage — see Task 5.2).

4. Model Size Selection and Kaggle Compute Budget

4.1. Choosing the Right Model Variant

YOLO models (v10, v12, v26) are released in five sizes: **n** (nano), **s** (small), **m** (medium), **l** (large), **x** (extra-large). RF-DETR is available as **nano** or **base**. Groups must choose one size per model based on their assigned dataset. The table below provides guidance; the choice must be justified in each model's CONFIG cell.

Dataset Images	YOLO Size	RF-DETR Size	Rationale
< 300	n	nano	Very small dataset; larger models will overfit severely. Regularise heavily.
300 – 1 000	n or s	nano	Prefer n for single-class; s only if 3+ classes and strong augmentation.
1 000 – 3 000	s or m	nano	Sufficient capacity for s; step to m if validation mAP plateaus early.
3 000 – 8 000	m	nano or base	m is the balanced default; nano RF-DETR trains faster with good results.
8 000 – 20 000	m or l	base	Higher model capacity is now justified; monitor Kaggle session time.
> 20 000	l	base	x is possible but risks session timeout at 50 epochs; benchmark first.

Note — YOLO x variant

YOLOv10/v12/v26-x can exceed Kaggle's single-session GPU limit (~9 hours) on medium-to-large datasets. If you select x, run a 1-epoch smoke test first, extrapolate the per-epoch wall time, and reduce epochs if necessary. Document the decision in the CONFIG cell.

4.2. Estimated Kaggle GPU Training Time (T4, single session)

Times below are approximate for 50 YOLO epochs / 20 RF-DETR epochs on Kaggle T4 GPU. Actual time depends on dataset size, image resolution, and batch size.

Size	<1 000 images	1 000–5 000 images	>5 000 images
<i>YOLO (v10, v12, v26) — minimum 50 epochs</i>			
n	15–30 min	30–60 min	60–110 min
s	25–50 min	50–100 min	100–180 min
m	50–90 min	90–160 min	160–280 min
l	80–140 min	140–260 min	260–420 min
x	130–220 min	220–400 min	Timeout risk
<i>RF-DETR — minimum 20 epochs</i>			
nano	20–45 min	45–90 min	90–160 min
base	50–100 min	100–200 min	200–380 min

Kaggle Session Strategy

- Always run a **1-epoch dry run** before committing to full training. Multiply per-epoch time by your target epoch count and verify it fits within ~8 hours (leave 1 hour buffer for evaluation and saving outputs).
- Save checkpoints to `/kaggle/working/` after every training run. Kaggle clears `/kaggle/working/` between sessions; download or attach as a dataset output before the session ends.
- Use `resume=True` (YOLO) or equivalent checkpoint-resume logic if a session times out mid-training.

4.3. Hyperparameter Tuning Recommendations

The following recommendations are starting points. Groups are expected to experiment and report what worked and what did not in each model notebook.

4.3.1. General (all models)

- **Input resolution:** Start at 640×640. Increase to 1280 only if your dataset has many small objects and VRAM allows. Document the resolution chosen and why.
- **Batch size:** Larger batches stabilise gradients but consume VRAM. Use the largest batch that fits without OOM; typical safe values for T4: batch 16 at 640, batch 4–8 at 1280.
- **Learning rate:** If doubling batch size, scale LR linearly ($\text{lr} \times \frac{\text{new_batch}}{\text{default_batch}}$).
- **Early stopping:** Set patience to 20–30 epochs (monitor mAP@50:95).
- **Warmup:** Use 3–5 warmup epochs to avoid unstable early gradients.

4.3.2. YOLO family (v10, v12, v26)

- **close_mosaic:** Disable mosaic augmentation for the last 10–15 epochs (`close_mosaic=10`) to improve final box precision.
- **flip_lr:** Set to 0.0 for datasets where left–right orientation is semantically meaningful (e.g., traffic signs, text detection).
- **degrees:** Keep at 0 for aerial/satellite imagery (objects are rotation-invariant) or increase to 10–20 for general scenes.
- **mixup / copy_paste:** Enable for multi-class datasets with >5 classes or severe class imbalance. Keep off for single-class datasets.
- **iou (NMS threshold):** Default 0.7 is appropriate for most cases; lower to 0.5–0.6 for dense object

scenes to reduce duplicate detections.

- **conf:** Set inference confidence to 0.25 during evaluation; increase to 0.4–0.5 if false positives dominate your error analysis.

4.3.3. RF-DETR

- **Backbone (frozen vs. fine-tuned):** The DINOv2 backbone is pre-trained and powerful. Freeze it for the first 5–10 epochs and then unfreeze; this prevents destroying pre-trained features early in training.
- **Gradient clipping:** Use `gradient_clip_max_norm=0.1` to stabilise transformer decoder training.
- **Learning rate (backbone vs. head):** Use a 10× lower LR for the backbone than for the detection head (e.g., backbone LR = 1×10^{-5} , head LR = 1×10^{-4}).
- **Resolution:** RF-DETR is sensitive to resolution; 560 or 640 is the recommended starting point.
- **Epochs:** 20 epochs is the minimum; RF-DETR typically converges faster than YOLO due to its stronger backbone features. Monitor validation AP closely.

4.3.4. Dataset-characteristic tuning

- **Dense objects / high overlap:** Increase `overlap_mask=True` in YOLO; reduce mosaic probability; focus error analysis on merged detections.
- **Small objects:** Increase input resolution; use multi-scale training (`scale=0.5`); evaluate mAP on small/medium/large split if COCO-style.
- **High aspect ratio images:** Avoid square-only crop augmentation; use `rect=True` in YOLO for rectangular inference.
- **Class imbalance:** Apply class-weighted loss or focal loss; over-sample rare classes during augmentation.

5. Assignment Tasks

The pipeline below maps to five notebooks. Tasks 5.1–5.4 belong to Notebook 1; Tasks 5.5–5.7 are repeated independently in Notebooks 2–5 (one per model).

5.1. Exploratory Data Analysis

Analyze the assigned dataset before touching any model:

- Total image count and count per existing split (if any).
- Class distribution: instance count per class; flag imbalance if present.
- Image properties: resolution distribution, aspect ratio, file format.
- Bounding-box statistics: count per image, width/height/area distributions, small vs. large object proportion ($< 32^2$, 32^2 – 96^2 , $> 96^2$ pixels).
- Sample annotated images (boxes overlaid) covering varying class densities.

5.2. Train / Validation / Test Splitting — Data Leakage Prevention

Grading Focus — Leakage

This is a specific grading criterion. Verify and document all points below before finalising any split.

- **Stratified class distribution** preserved across splits; report per-split class counts after splitting.

- **Justified split ratio** (70/15/15 or 80/10/10) with reasoning stated in the notebook given your dataset size.

5.3. Data Augmentation

Design an augmentation pipeline appropriate to your dataset's domain and the failure modes identified in EDA. Each transform must be individually justified:

- Geometric: flips, rotation, scale/crop, mosaic (with `close_mosaic` setting).
- Photometric: brightness/contrast/HSV jitter, blur, noise — only where imaging condition variation is present in the dataset.
- Verify bounding boxes remain correctly aligned after every transform.

5.4. Input Pipeline Verification

- Plot a batch of augmented images with ground-truth boxes overlaid for each model's expected input format.
- Confirm box-to-image alignment after resizing and augmentation.
- Identify and fix flipped boxes, coordinate errors, or class-index mismatches *before* training — not during.

5.5. Model Training (one notebook per model)

Each model notebook trains one detector on the splits from Notebook 1, using the size selected per Section 4.1. Minimum epochs: **50 for YOLO, 20 for RF-DETR**. All CONFIG parameters must be declared at the top of the notebook.

5.6. Testing and Inference (per model notebook)

- Run inference on the **held-out test split only**.
- Report: mAP@50, mAP@50:95, Precision, Recall, F1-score; per-class metrics if multi-class.
- Report inference speed (FPS) measured on the same hardware.

5.7. Error Analysis (per model notebook)

- At least **3 false negative** examples with written cause analysis (small size, occlusion, low contrast, out-of-distribution appearance, etc.).
- At least **3 false positive** examples with cause analysis.
- Localization errors and misclassifications (if multi-class).
- A written **failure-pattern summary**: one dominant failure mode for this model on this dataset, grounded in the examples shown.

6. Required Notebook Structure

Each group submits exactly **five** Kaggle notebooks. Every notebook must open with a CONFIG cell declaring all paths, class names, model size, and key hyperparameters — the CONFIG cell is the first thing inspected during the viva.

6.1. Notebook 1 — Dataset Exploration & Preprocessing

Covers Tasks 5.1–5.4.

1. Title cell: group number, member names and IDs, dataset name and source link.
2. CONFIG cell: dataset paths, annotation format, class names, image size, target split ratios.
3. Dataset integrity check: file count, missing or corrupt files, format consistency.
4. EDA: class distribution chart, image resolution histogram, bounding-box size/count distributions, annotated sample images (Task 5.1).
5. Split creation with MD5/hash leakage check, stratification report, and written split-ratio justification (Task 5.2).
6. Augmentation pipeline: each transform listed with a one-sentence justification tied to the EDA findings (Task 5.3).
7. Before/after augmentation visualisation with boxes overlaid; confirm alignment (Task 5.4).
8. Written summary: 3–5 sentences on dataset characteristics that will influence model-size and hyperparameter choices.

Viva Questions — Notebook 1

1. What does your class distribution plot reveal about this dataset, and how did it specifically change your augmentation choices?
2. Walk through your leakage check. What method did you use, what did you find, and why is your split now safe?
3. Justify your chosen split ratio given the number of images in your dataset. What would happen if you allocated more data to the test set?
4. You applied *[augmentation X]*. Why is it appropriate for this dataset? Name one augmentation you deliberately excluded and why.
5. Show your before/after augmentation visualisation. Point to one image and confirm the box coordinates are correct after the transform.
6. Your EDA shows a bounding-box size distribution skewed toward small objects. How does this affect your choice of model size and input resolution?

6.2. Notebook 2 — YOLOv10: Training, Evaluation & Error Analysis

Covers Tasks 5.5–5.7 for YOLOv10.

1. Title cell: group and dataset reference consistent with Notebook 1.
2. CONFIG cell: model variant size chosen (n/s/m/l/x), justification, epochs (≥ 50), image size, batch size, optimizer, LR and schedule, augmentation parameters, NMS thresholds.
3. Environment setup; `data.yaml` generation from Notebook 1 splits.
4. 1-epoch dry run to confirm pipeline correctness and estimate total training time.
5. Full training run with logged loss curves and mAP@50 / mAP@50:95 curves.
6. Test-split inference: mAP@50, mAP@50:95, Precision, Recall, F1, FPS (Task 5.6).

7. Error analysis: ≥ 3 false negatives, ≥ 3 false positives, written failure-pattern summary (Task 5.7).
8. Save best checkpoint to `/kaggle/working/` with a clearly named path recorded in the notebook for cross-model comparison.

Viva Questions — Notebook 2 (YOLOv10)

1. You chose YOLOv10-*[size]*. Why this size for your dataset? What would change if you selected the next size up or down?
2. What learning rate did you use? How did you choose it, and did you use a scheduler? Show the LR curve if available.
3. Your training loss curve shows *[pattern]*. What does this indicate about convergence, and would you train for more epochs?
4. What is the difference between mAP@50 and mAP@50:95? Which metric better captures detection quality for your specific dataset and why?
5. Walk through one false negative. What do you believe caused the missed detection, and what hyperparameter or augmentation change might address it?
6. You set `conf=0.25` for inference. What happens to Precision and Recall if you increase this to 0.5? Show the trade-off.
7. What is the role of NMS in your inference pipeline, and how does the IoU threshold affect the number of detections?

6.3. Notebook 3 — YOLOv12: Training, Evaluation & Error Analysis

Covers Tasks 5.5–5.7 for YOLOv12.

1. Title cell: group and dataset reference consistent with Notebook 1.
2. CONFIG cell: model variant size (n/s/m/l/x), justification, epochs (≥ 50), and all hyperparameters. Note any intentional differences from YOLOv10 config.
3. Environment setup; re-use `data.yaml` from Notebook 2 or regenerate.
4. 1-epoch dry run to confirm pipeline.
5. Full training run with loss and mAP curves.
6. Test-split inference: mAP@50, mAP@50:95, Precision, Recall, F1, FPS (Task 5.6).
7. Error analysis: ≥ 3 false negatives, ≥ 3 false positives, written failure-pattern summary; note any differences from YOLOv10 failure patterns (Task 5.7).
8. Save best checkpoint.

Viva Questions — Notebook 3 (YOLOv12)

1. What architectural change distinguishes YOLOv12 from YOLOv10? How does this difference appear in the training behaviour?
2. Did you use the same hyperparameters as YOLOv10? If you changed anything, explain what and why.
3. Compare your YOLOv12 and YOLOv10 training curves side-by-side. Which converged faster? Which reached a higher final mAP?
4. Show a false positive from YOLOv12. Does YOLOv10 also produce this error on the same image? What does that tell you?
5. You have both mAP@50 and FPS for YOLOv10 and YOLOv12. For a real-time deployment scenario, which model would you choose and why?
6. What is `close_mosaic`? Did you use it? What effect did it have on your final box precision?

6.4. Notebook 4 — YOLOv26: Training, Evaluation & Error Analysis

Covers Tasks 5.5–5.7 for YOLOv26.

1. Title cell: group and dataset reference consistent with Notebook 1.
2. CONFIG cell: model variant size (n/s/m/l/x), justification, epochs (≥ 50), and all hyperparameters. Note any differences from previous YOLO configs and the reasoning.
3. Environment setup; re-use or regenerate `data.yaml`.
4. 1-epoch dry run.
5. Full training run with loss and mAP curves.
6. Test-split inference: mAP@50, mAP@50:95, Precision, Recall, F1, FPS (Task 5.6).
7. Error analysis: ≥ 3 false negatives, ≥ 3 false positives, written failure-pattern summary; compare against YOLOv10 and YOLOv12 patterns (Task 5.7).
8. Save best checkpoint.

Viva Questions — Notebook 4 (YOLOv26)

1. What does YOLOv26 improve upon compared to YOLOv10 and YOLOv12? Did you observe this improvement in your results?
2. You trained all three YOLO variants on the same dataset. Which performed best by mAP@50:95? Does the ranking change when you consider FPS?
3. Show a detection that YOLOv26 gets right but one of the earlier models missed (or vice versa). What explains the difference?
4. How did the training time of YOLOv26 compare to YOLOv10 at the same size? Was the accuracy gain worth the extra compute?
5. Your failure-pattern summary states *[pattern]*. What augmentation or architectural change might address this failure mode?
6. If you could only submit one YOLO model to production for this dataset, which size and version would you choose and why?

6.5. Notebook 5 — RF-DETR: Training, Evaluation & Error Analysis

Covers Tasks 5.5–5.7 for RF-DETR.

1. Title cell: group and dataset reference consistent with Notebook 1.
2. CONFIG cell: RF-DETR variant (nano or base), justification, epochs (≥ 20), input resolution, backbone LR vs. head LR, gradient clip norm, warmup, and frozen-backbone epochs.
3. Environment setup; convert Notebook 1 splits to the format expected by RF-DETR (COCO JSON if not already).
4. 1-epoch dry run to estimate training time.
5. Full training run with loss components (classification loss, box regression loss, contrastive/matching loss) and mAP curves logged.
6. Test-split inference: mAP@50, mAP@50:95, Precision, Recall, F1, FPS (Task 5.6).
7. Error analysis: ≥ 3 false negatives, ≥ 3 false positives, written failure-pattern summary; note qualitative differences from YOLO failure patterns (Task 5.7).
8. Save best checkpoint.

Viva Questions — Notebook 5 (RF-DETR)

1. What is the fundamental architectural difference between DETR-based detection and YOLO-based detection? How does this affect training time and inference?
2. You used RF-DETR-*[nano/base]*. Why did you choose this variant for your dataset? What is the main trade-off between nano and base?
3. Your CONFIG cell sets a backbone LR of X and a head LR of Y . Why are these different? What happens if you use the same LR for both?
4. RF-DETR uses a DINOv2 backbone. What is DINOv2 and why is a pre-trained backbone especially valuable for small datasets?
5. Explain the role of the bipartite matching loss in RF-DETR. Why does DETR not require NMS post-processing?
6. Comparing RF-DETR to your best YOLO model: which has higher mAP@50:95? Which is faster? Which would you deploy and under what conditions?
7. Show a failure case where RF-DETR misses an object that YOLO detects. What might explain this?

7. Submission Guidelines

1. All five notebooks must run end-to-end on Kaggle with every cell output visible — no missing outputs, no incomplete runs.
2. Set each notebook's Kaggle visibility to **Public** and commit the final executed version before copying the link.
3. Post one submission on Google Classroom per group containing: group number, member names and IDs, dataset name, and five public Kaggle notebook links clearly labelled Notebook 1–5.
4. Deadline: **July 12, 2026 (12.07.2026)**.
5. Test all five links in a private/incognito browser window before submitting; private or broken links are treated as missing notebooks.

8. Viva Voce Examination

Assessment is conducted primarily through a viva voce examination. The viva focuses on the code and parameters used, not on final metric numbers.

8.1. Format

- Conducted per group after the submission deadline; schedule announced on Google Classroom.
- All group members must attend and must answer independently — a single member cannot speak for the whole group.
- Open-notebook: students may have notebooks open, but answers must demonstrate understanding, not reading code aloud.

8.2. General Focus Areas

- Justify the model size chosen in the CONFIG cell of any notebook against the dataset characteristics identified in Notebook 1.
- Explain the data-leakage prevention method in Notebook 1 and why it is sufficient for this dataset.
- Connect augmentation choices to specific EDA observations.
- Explain how mAP@50:95 is computed and what it captures that mAP@50 does not.
- Walk through at least one error-analysis example from each model notebook and explain the likely cause.
- Explain why RF-DETR uses a lower learning rate for the backbone than for the detection head.

Note

Code that a student cannot explain — including code taken from tutorials or from other groups — is treated as not understood, regardless of whether it runs correctly. This applies to every line in the CONFIG cell and every parameter passed to the training API.

9. Grading Rubric

Component	Weight
Notebook 1 — EDA, Leakage-Safe Splitting & Augmentation	15%
Notebook 2 — YOLOv10: Training, Evaluation & Error Analysis	15%
Notebook 3 — YOLOv12: Training, Evaluation & Error Analysis	15%
Notebook 4 — YOLOv26: Training, Evaluation & Error Analysis	15%
Notebook 5 — RF-DETR: Training, Evaluation & Error Analysis	15%
Model-size justification and tuning rationale (all notebooks)	10%
Viva Voce — Code, Parameters & Results Understanding	15%
Total	100%

10. Important Notes

- This document covers Part 1 only; Part 2 will be announced on Google Classroom.
- Use only your assigned dataset across all five notebooks.
- Do not share checkpoints, code, or notebooks between groups — similarity is checked and discussed in the viva.
- If a dataset constraint makes a requirement impractical (e.g., too few images for a 70/15/15 split), document the issue and your adapted approach in Notebook 1; do not deviate silently.
- Every CONFIG-cell parameter must be justifiable in the viva. Parameters copied from tutorials without understanding will be probed directly.
- Direct questions about this assignment to the instructor via Google Classroom.

Assignment 1 — Key Requirements at a Glance

1. **Five notebooks:** EDA & Preprocessing → YOLOv10 → YOLOv12 → YOLOv26 → RF-DETR (each model notebook includes training, evaluation, and error analysis).
2. **Model size** (n/s/m/l/x for YOLO; nano/base for RF-DETR) chosen by the group based on dataset size and justified in the CONFIG cell.
3. Minimum **50 epochs** for YOLO models; **20 epochs** for RF-DETR. Run a 1-epoch dry run first to verify Kaggle session time budget.
4. **Data-leakage checks** (MD5/hash) completed before any split is used.
5. Deadline: **July 12, 2026** — five public Kaggle links via Google Classroom.
6. Assessment is viva-based: each member must justify *every* CONFIG parameter and explain at least one error-analysis case per model.

Questions: Post on the course Google Classroom stream.

Next: Part 2 of this assignment — details to be announced.